

Light of Islam — Azan Reminder Push API

Full Requirements Document

Field	Value
Product	Light of Islam (mobile app)
Component	Backend API & push scheduling service
Document version	1.0
Date	30 September 2026
Prepared for	Zippy Software House / Light of Islam team

1. Purpose

This document defines the complete requirements for the **Light of Islam Azan Reminder** backend: a service that registers mobile devices and sends **Firebase Cloud Messaging (FCM)** push notifications at each daily prayer time (Fajr, Dhuhr, Asr, Maghrib, Isha) based on the device's geographic location and timezone.

The mobile app does **not** send prayer times to the server. The backend is solely responsible for resolving prayer times, scheduling, and delivery.

2. Goals & success criteria

2.1 Goals

- Deliver reliable azan reminder pushes at the correct local time per device.
- Keep the mobile client simple: register once (or on token/location change).
- Use **data-only** FCM messages so the Android app controls notification channel and azan sound.
- Support Pakistan prayer calculation via Aladhan **method 1** (Karachi / University of Islamic Sciences, Karachi).

2.2 Success criteria

- A registered device receives up to **five** prayer reminders per day at times matching Aladhan for its coordinates and timezone.
 - Production reminders use `data.type = "azan_reminder"` so the app plays the azan sound.
 - Test pushes from Firebase Console (without custom data) do **not** play the azan sound.
 - Device registration API is available over HTTPS on a dedicated subdomain (recommended: `api.lightislam.com`).
-

3. Scope

3.1 In scope

- Device registration (FCM token + location + timezone).
- Persistent storage of device records.
- Daily (and on-registration) fetch of prayer times from Aladhan API.
- Job queue / scheduler for delayed push delivery.
- FCM integration via Firebase Admin SDK.

- Health check endpoint for monitoring.
- Environment-based configuration (no committed secrets).

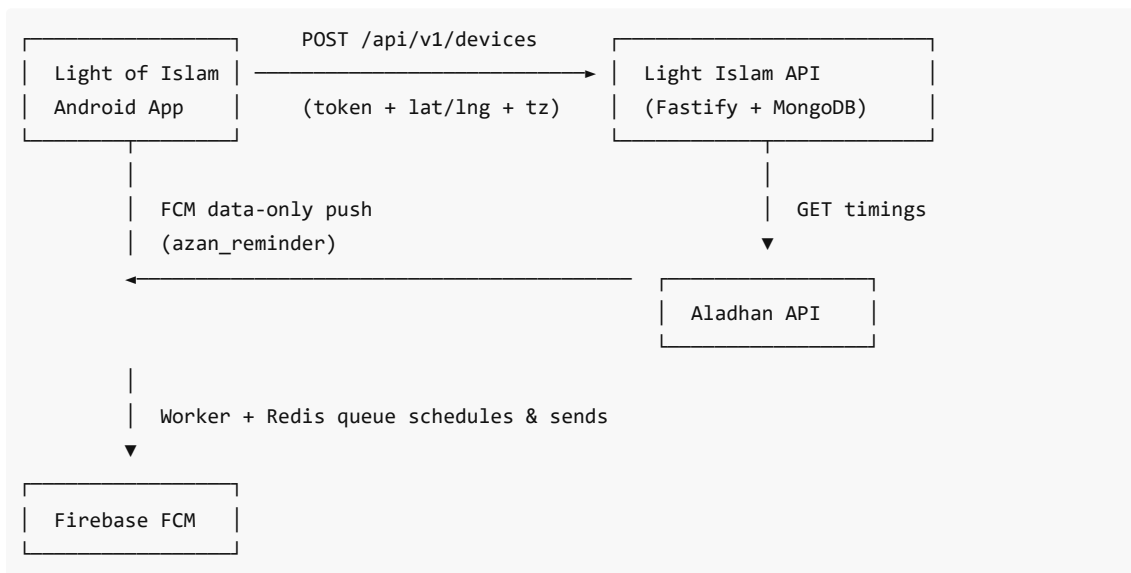
3.2 Out of scope (current phase)

- User accounts, login, or OAuth for the mobile app calling this API.
- Admin dashboard for devices.
- iOS-specific requirements beyond standard FCM data payloads (app team owns client handling).
- Storing or serving Quran, ads, or other app content.
- Prayer times posted by the client.

3.3 Optional / future

- Registration using **city/country** instead of latitude/longitude.
- Public OpenAPI (Swagger) documentation UI.
- Device unregister API.
- Per-device timezone midnight re-schedule (vs single UTC cron).

4. System context



Division of responsibility

Layer	Responsibility
Client	Obtain FCM token; read GPS or fixed location; send registration payload; handle incoming data messages and notification channels.
API server	Accept registration; persist device; enqueue scheduling work.
Worker	Fetch prayer times; schedule delayed jobs; send FCM; remove invalid tokens.
Aladhan	Authoritative prayer timings for lat/lng, date, method, timezone string.
Firebase	Deliver push to device.

5. Client (mobile app) requirements

5.1 When to call the backend

The app **must** call device registration when:

1. App starts (if token or location available).
2. FCM token is refreshed.
3. User location or timezone changes materially.

5.2 What the client must send

The client **must only** send registration data. It **must not** POST prayer times or schedules.

5.3 What the client must not do

- Must not rely on local alarms alone for production azan reminders if product requirement is server-driven push.
- Must not expect a `notification` payload from the server for azan reminders (see section 8).

5.4 Android notification channels

The app **must** implement two notification channels:

Channel	Sound	Used when
Azan channel	Azan audio	<code>data.type == "azan_reminder"</code>
Default channel	System default	Any other or missing type

This ensures manual Firebase Console test messages do not trigger azan audio.

6. Backend functional requirements

FR-1 Device registration

The system **shall** expose an HTTP endpoint to create or update a device record keyed by FCM token.

FR-2 Device storage

The system **shall** store per device:

- FCM token (unique).
- Latitude and longitude (required for current implementation).
- IANA timezone string (e.g. `Asia/Karachi`).
- Created/updated timestamps.

Optional future: city and country instead of coordinates.

FR-3 Prayer time resolution

For each device, the system **shall** obtain daily prayer timings by calling Aladhan:

```
GET https://api.aladhan.com/v1/timings/{DD-MM-YYYY}?latitude={X}&longitude={Y}&method=1&timezonestring={Timezone}
```

- **method=1**: Karachi convention (Pakistan).
- **Prayers**: Fajr, Dhuhr, Asr, Maghrib, Isha.
- Times in response `data.timings` are 24-hour strings (e.g. `"05:12"`).

No API key is required for Aladhan public timings endpoint.

FR-4 Scheduling

The system **shall**:

1. Schedule **one push per prayer time per device** for the relevant day(s).
2. **Re-fetch and re-schedule daily** because prayer times shift.
3. Trigger scheduling when a device registers or updates.
4. Use a durable queue (e.g. Redis + BullMQ) for delayed delivery at exact prayer times.

FR-5 Push delivery

At each scheduled time, the system **shall** send an FCM message to the device token using Firebase Admin SDK.

FR-6 Invalid token handling

If FCM returns an invalid or unregistered token, the system **shall** remove that device record to avoid repeated failures.

FR-7 Health

The system **shall** expose `GET /health` returning success when the API process is running.

7. API specification

7.1 Base URL

Environment	Recommended host
Production	<code>https://api.lightislam.com</code>
Staging	<code>https://api-staging.lightislam.com</code> (optional)

7.2 Register or update device

Endpoint: `POST /api/v1/devices`

Authentication: None (current phase). Rate limiting recommended for production.

Request body (JSON):

Field	Type	Required	Description
<code>fcm_token</code>	string	Yes	Current FCM registration token
<code>latitude</code>	number	Yes	-90 to 90
<code>longitude</code>	number	Yes	-180 to 180
<code>timezone</code>	string	Yes	IANA timezone, e.g. Asia/Karachi

Example:

```
{
  "fcm_token": "dGhpcyBpcyBhbiBleGFtcGx1IHRva2Vu",
  "latitude": 33.6844,
  "longitude": 73.0479,
  "timezone": "Asia/Karachi"
}
```

Success response: 201 Created

```
{
  "ok": true,
  "device_id": "507f1f77bcf86cd799439011"
}
```

Validation errors: 400 with clear message (recommended).

Behaviour: Upsert by `fcm_token` ; enqueue prayer scheduling for that device.

7.3 Health check

Endpoint: GET /health

Success response: 200 OK

```
{
  "ok": true
}
```

8. Push notification requirements (FCM)

8.1 Data-only messages

Messages **must** contain **only** a `data` object. There **must be no** top-level `notification` key.

If `notification` is present, Android may render the notification in the system tray with the default sound and bypass app channel logic.

8.2 Required data fields (production azan)

Key	Required	Type	Example	Notes
type	Yes	string	azan_reminder	Exact value required for azan sound
prayerName	Yes	string	Fajr	One of five prayer names
time	Yes	string	05:12	24h local time string
title	No	string	Fajr	May be derived on client
body	No	string	Time for Fajr prayer	May be derived on client

All `data` values must be strings. FCM rejects non-string values in the data map.

8.3 Reference implementation (Node.js)

```
await admin.messaging().send({
  token: deviceFcmToken,
  data: {
    type: 'azan_reminder',
    title: 'Fajr',
    body: 'Time for Fajr prayer',
    prayerName: 'Fajr',
    time: '05:12'
  }
});
```

8.4 Sound isolation rule

data.type	App behaviour
azan_reminder	Azan notification channel
Missing or any other value	Default notification channel

9. Data model

9.1 Device collection (MongoDB)

Field	Type	Constraints
fcmToken	string	Unique, indexed
latitude	number	Required
longitude	number	Required
timezone	string	Required
createdAt	date	Auto
updatedAt	date	Auto

10. Technical architecture (implementation reference)

Component	Technology
Runtime	Node.js 20+
Language	TypeScript
HTTP framework	Fastify 5
Database	MongoDB (Mongoose)
Queue	BullMQ + Redis

Date/time	Luxon (timezone-aware scheduling)
Push	firebase-admin

10.1 Processes

1. **API server** — HTTP routes, MongoDB connection, enqueue scheduler jobs.
2. **Worker** — Consumes queue, calls Aladhan, sends FCM, deletes bad tokens.

Both processes require MongoDB and Redis. Worker requires Firebase environment variables.

10.2 Daily scheduler

A repeatable job **shall** re-schedule all devices at least once per day (implementation: cron pattern on scheduler queue). Ideal enhancement: align with each device's local midnight.

11. Configuration & secrets

Configuration via environment variables only. **Do not** commit `.env` or service account JSON files to source control.

Variable	Required by	Description
PORT	API	HTTP port (default 3000)
HOST	API	Bind address (default 0.0.0.0)
MONGODB_URI	API, Worker	MongoDB connection string
REDIS_URL	API, Worker	Redis connection for BullMQ
FIREBASE_PROJECT_ID	Worker	Firebase project ID
FIREBASE_CLIENT_EMAIL	Worker	Service account client email
FIREBASE_PRIVATE_KEY	Worker	PEM private key; use \n for line breaks in <code>.env</code>

12. Non-functional requirements

12.1 Availability

- API and worker should be deployed with process restart on failure.
- Redis and MongoDB should use managed or replicated setups in production.

12.2 Performance

- Registration endpoint should respond in under 500 ms under normal load (excluding queue enqueue).
- Aladhan and FCM calls occur in worker context, not blocking registration.

12.3 Security

- HTTPS only in production.
- Protect Firebase and database credentials.
- Consider API rate limiting and optional API key in a later phase.

12.4 Observability

- Structured logging on API and worker.
- Monitor queue depth, job failures, and FCM error rates.

12.5 Privacy

- Store only data needed for prayer notifications (token, location, timezone).
- Document retention and deletion policy for production launch.

13. External dependencies

Service	Purpose	SLA / notes
Aladhan API	Prayer timings	Public HTTP; implement retries on failure
Google Firebase FCM	Push delivery	Requires valid service account
MongoDB	Device persistence	
Redis	Job queue	Required for scheduled pushes

14. Acceptance checklist

- ☐ App can register device with sample payload; receives `201` and `device_id` .
- ☐ Device appears in MongoDB with correct fields.
- ☐ Worker schedules jobs and sends FCM at prayer time (test with near-future schedule).
- ☐ FCM payload is data-only with `type: azan_reminder` and all string fields.
- ☐ App plays azan sound only for `azan_reminder` type.
- ☐ Manual Firebase test push without `type` uses default sound only.
- ☐ Invalid FCM token removes device from database.
- ☐ Daily re-schedule runs for all devices.
- ☐ Production deployed at `https://api.lightislam.com` (or agreed subdomain) with TLS.

15. Glossary

Term	Definition
FCM	Firestore Cloud Messaging
Azan	Islamic call to prayer; app plays dedicated audio for reminders
IANA timezone	Standard names such as Asia/Karachi
Data-only message	FCM message with data map only, no notification block
Upsert	Create or update existing record by unique key (FCM token)

16. Document history

Version	Date	Changes
---------	------	---------

1.0	30 Sep 2026	Initial full requirements PDF source
-----	-------------	--------------------------------------

End of document